

DELEGATE - ỦY QUYỀN

1

1

Delegate

- ❑ delegate là kiểu dữ liệu đặc biệt, có khả năng lưu trữ một tham chiếu tới phương thức có dấu hiệu cụ thể (tức là các tham số và kiểu trả về của phương thức).
- ❑ delegate có thể viện dẫn các phương thức một cách động khi sự kiện xảy ra.
- ❑ delegate không quan tâm tới những lớp đối tượng mà nó tham chiếu tới.

2

2

delegate LÀ KIỂU PHƯƠNG THỨC

Khai báo kiểu delegate

```
delegate void Notifier (string sender); // tương tự khai báo phương thức
// sử dụng từ khoá delegate
```

Khai báo biến delegate

```
Notifier greetings;
```

Gán một phương thức cho biến delegate

```
void SayHello(string sender) {
    Console.WriteLine("Hello from " + sender);
}
```

```
greetings = new Notifier(SayHello);
```

Lời gọi biến delegate

```
greetings("John"); // viện dẫn phương thức SayHello("John") =>
                    "Hello from John"
```

3

3

GÁN PHƯƠNG THỨC KHÁC NHAU

- ❑ **Tất cả các phương thức phù hợp với delegate đều có thể được gán với biến delegate đó**

```
void SayGoodBye(string sender) {
    Console.WriteLine("Good bye from " + sender);
}
greetings = new Notifier(SayGoodBye);
greetings("John"); // SayGoodBye("John") => "Good bye from John"
```

Chú ý

- ❑ Biến delegate có thể được gán giá trị *null* (không có phương thức nào được gán cho nó).
- ❑ Nếu biến delegate bằng *null* thì sẽ không được gọi

4

4

TẠO GIÁ TRỊ CỦA delegate

`new DelegateType (obj.Method)`

- ❑ Biến delegate chứa phương thức và đối tượng nhận, nhưng không chứa tham số
`new Notifier(myObj.SayHello);`
- ❑ Đối tượng có thể là *this* (và có thể bỏ qua)
`new Notifier(SayHello);`
- ❑ Phương thức có thể là *static*. Trong trường hợp này, tên của class phải được thay thế cho đối tượng.
`new Notifier(MyClass.StaticSayHello);`
- ❑ Phương thức không thể là *abstract*, nhưng có thể là *virtual*, *override*, hoặc *new*.
- ❑ Dấu hiệu của phương thức phải trùng với dấu hiệu của *DelegateType*
 - số lượng tham số
 - kiểu dữ liệu của tham số (bao gồm cả kiểu trả về)
 - kiểu truyền tham số (ref, out, value)

5

5

MULTICAST delegate

- ❑ **Biến delegate có thể chứa nhiều giá trị cùng một thời điểm**

```
Notifier greetings;
greetings = new Notifier(SayHello);
greetings += new Notifier(SayGoodBye);

greetings("John");           // "Hello from John"
                             // "Good bye from John"

greetings -= new Notifier(SayHello);
greetings("John");           // "Good bye from John"
```

- ❑ **Chú ý**
 - Nếu multicast delegate là một hàm, thì sẽ trả về giá trị của lời gọi cuối cùng
 - Nếu multicast delegate có tham số out, tham số của lời gọi cuối cùng sẽ được trả về. Tham số ref được truyền qua tất cả các phương thức.

6

6

Event là delegate đặc biệt

```
class Model {
    public event Notifier notifyViews;
    public void Change() { ... notifyViews("Model"); }
}

class View {
    public View(Model m) { m.notifyViews += new Notifier(Update); }
    void Update(string sender) { Console.WriteLine(sender + " was changed"); }
}

class Test {
    static void Main() {
        Model model = new Model();
        new View(model); new View(model); ...
        model.Change();
    }
}
```

Tại sao phải sử dụng event để thay thế cho các biến delegate khác?

- ☐ Chỉ class khai báo event thì mới được kích hoạt nó
- ☐ Các class khác có thể thay đổi event bằng toán tử += hoặc -= (nhưng không sử dụng toán tử =)

7

7

BẮT SỰ KIỆN

Các delegate dùng để bắt sự kiện có dấu hiệu như sau:

```
delegate void SomeEvent (object source, MyEventArgs e);
```

- ☐ Kiểu trả về: void
- ☐ Tham số thứ nhất: Đối tượng gửi sự kiện (kiểu *object*)
- ☐ Tham số thứ hai: sự kiện (subclass của *System.EventArgs*)

```
public class EventArgs {
    public static readonly EventArgs Empty;
}
```

8

8

VÍ DỤ

```
public delegate void KeyEventHandler (object sender, EventArgs e);

public class EventArgs : EventArgs {
    public virtual bool Alt { get {...} } // true nếu nhấn phím Alt
    public virtual bool Shift { get {...} } // true nếu nhấn phím Shift
    public bool Control { get {...} } // true nếu nhấn phím Ctrl
    public bool Handled { get {...} set {...} } // cho biết có bắt được sự
    kiện không
    public int KeyValue { get {...} } // mã phím
    ...
}

class MyKeyEventSource {
    public event KeyEventHandler KeyDown;
    public KeyPressed() {
        KeyDown(this, new EventArgs(...));
    }
}

class MyKeyListener {
    public MyKeyListener(...) { keySource.KeyDown += new
    KeyEventHandler(HandleKey); }
    void HandleKey (object sender, EventArgs e) {...}
}
```

9